SMART RESUME SCREENING TOOL

A Project Based Learning Report Submitted in Partial Fulfilment of the Requirements for the Award of the Degree of

of

Bachelor of Technology

in The Department of Artificial Intelligence and Data Science

Data Structures and Algorithms 25CS2103E

Submitted by

2510080029 - Oddesar Sudeeptha

2510080037 - Lokrey Latika

2510080043 - Samineni Sai Siri

2510080061 - Muvva Kovidha

Under the guidance of

Dr.P.Anita



Department of Artificial Intelligence and Data Science

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

July - 2026

ABSTRACT

The Smart Resume Screening Tool is a Java-based application designed to automate the initial screening and ranking of job applicants. Traditional resume screening requires recruiters to manually examine numerous resumes and compare candidate qualifications with job requirements. This process can be time-consuming and may lead to inconsistent evaluations.

The proposed system allows users to select a job description file and a folder containing multiple resume files through a graphical user interface developed using Java Swing. The application processes the selected resumes, extracts candidate information such as name, technical skills, years of experience, and certifications, and evaluates candidates according to the requirements specified in the job description.

The system uses data structures and algorithmic techniques to organize and rank candidates. Skill and experience matching contribute to candidate scoring, while a custom Max Heap is used to retrieve candidates in descending order of score. Java file handling supports reading job descriptions and resumes, and the graphical interface presents the results in an understandable format.

The project demonstrates the practical application of data structures and algorithms in recruitment automation. It aims to reduce manual screening effort, improve consistency in initial candidate evaluation, and provide recruiters with a ranked list of candidates for further review. The scores are intended to support, rather than replace, human recruitment decisions.

List of Figures

Figure No.	Title of the Figure	Page No.
Figure 2.1	System Workflow of Smart Resume Screening Tool	11
Figure 2.2	System Architecture of Smart Resume Screening Tool	11
Figure 3.1	Graphical User Interface of the Application	16
Figure 3.2	Job Description File Selection	17
Figure 3.3	Resume Folder Selection	17
Figure 5.1	Application Output	24

List of Tables

Table No	Title of the Table	Page No.
Table 1.1	Project Modules	10
Table 3.1	Software Requirements	15
Table 3.2	Hardware Requirements	15
Table 3.3	Algorithms and Time Complexity	19
Table 4.1	Candidate Data Fields	23
Table 5.1	Test Cases and Results	26-27

Table of Contents

S. No.	Contents	Page No.
	Abstract	2
	List of Figures	3
	List of Tables	4
	Table of Contents	5-7
1	**CHAPTER 1: INTRODUCTION**	8-10
1.1	Introduction	8
1.2	Problem Statement	9
1.3	Objectives of the Project	9
1.4	Scope of the Project	10
1.5	Project Modules	10
2	**CHAPTER 2: METHODOLOGY AND SYSTEM DESIGN**	11-14
2.1	System Workflow	11
2.2	System Architecture	11
2.3	Job Description Processing Module	12

2.4	Resume Parsing Module	12
2.5	Skill Matching and Candidate Scoring	13
2.6	Candidate Ranking using Max Heap	13
2.7	Graphical User Interface Module	14
3	**CHAPTER 3: IMPLEMENTATION**	15-19
3.1	Software Requirements	15
3.2	Hardware Requirements	15
3.3	Graphical User Interface	16-17
3.4	Resume Parsing Implementation	18
3.5	Candidate Scoring Implementation	18
3.6	Max Heap Implementation	18
3.7	Algorithms Analysis	19
4	**CHAPTER 4: DATA AND FILE DESIGN**	20-23
4.1	Job Description File	20
4.2	Resume File Structure	21-22
4.3	Candidate Data Structure	23
4.4	Max Heap Design	23

5	**CHAPTER 5: RESULTS AND DISCUSSION**	24-27
5.1	Results Obtained	24
5.2	Discussion	25
5.3	Advantages	25
5.4	Limitations	26
5.5	Testing Results	26-27
6	**CHAPTER 6: CONCLUSION AND FUTURE SCOPE**	28-29
6.1	Conclusion	28
6.2	Future Scope	29
	References	30

SMART RESUME SCREENING TOOL

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Recruitment is an important process in organizations, where identifying suitable candidates from a large number of applications requires significant time and effort. Recruiters must examine resumes, identify relevant skills, evaluate professional experience, and compare candidate qualifications with job requirements.

The Smart Resume Screening Tool aims to simplify the initial recruitment screening process by automating resume parsing, candidate scoring, and candidate ranking. The application is developed in Java and uses data structures and algorithms to process candidate information and organize the results.

Users can select a job description text file and a folder containing multiple resume text files. The system extracts relevant information from each resume, compares candidate qualifications with the job requirements, calculates a score, and ranks the candidates using a Max Heap.

The project demonstrates how concepts such as arrays, string processing, file handling, algorithmic scoring, and heap-based priority ordering can be applied to a practical recruitment problem.

1.2 Problem Statement

- Manual resume screening becomes difficult when the number of applicants increases.
- Recruiters must repeatedly compare candidate skills with job requirements.
- Identifying relevant experience and certifications across multiple resumes requires time.
- Manual comparisons can produce inconsistent shortlisting decisions.
- There is a need for a system that extracts candidate information, calculates a job-specific matching score, and ranks applicants efficiently.

1.3 Objectives

- To develop a Java-based resume screening application.
- To allow users to select a job description and resume folder through a GUI.
- To extract candidate names, skills, experience, and certifications from resume text files.
- To compare extracted candidate information with job requirements.
- To calculate a score for each candidate using a scoring algorithm.
- To implement a custom Max Heap for ranking candidates.
- To display candidate rankings and relevant information in a user-friendly interface.
- To demonstrate the practical use of DSA concepts in recruitment automation.

1.4 Scope of the Project

The current project focuses on screening .txt resume files against a text-based job description. It supports processing multiple resumes in one run and displaying ranked candidates in a Java Swing interface.

The current implementation is a prototype for initial screening. PDF and DOCX extraction, database integration, online recruitment, machine-learning-based evaluation, and cloud deployment are possible future enhancements unless they have been implemented separately.

1.5 Project Modules

Module	Description
Job Description Processing	Reads required skills, experience, and certifications from the selected job description.
Resume Parsing	Extracts candidate details from resume text files.
Skill Matching	Compares candidate skills with the required skills.
Candidate Scoring	Calculates a score based on configured matching criteria.
Candidate Ranking	Uses a custom Max Heap to retrieve candidates in descending score order.
File Handling	Reads the selected job description and resume files.
Graphical User Interface	Provides file selection, an analysis button, and a results display.

Table 1.1 : Project Modules

Chapter 2: Methodology

2.1 System Workflow



Figure 2.1 : System Workflow of Smart Resume Screening Tool

2.2 System Architecture

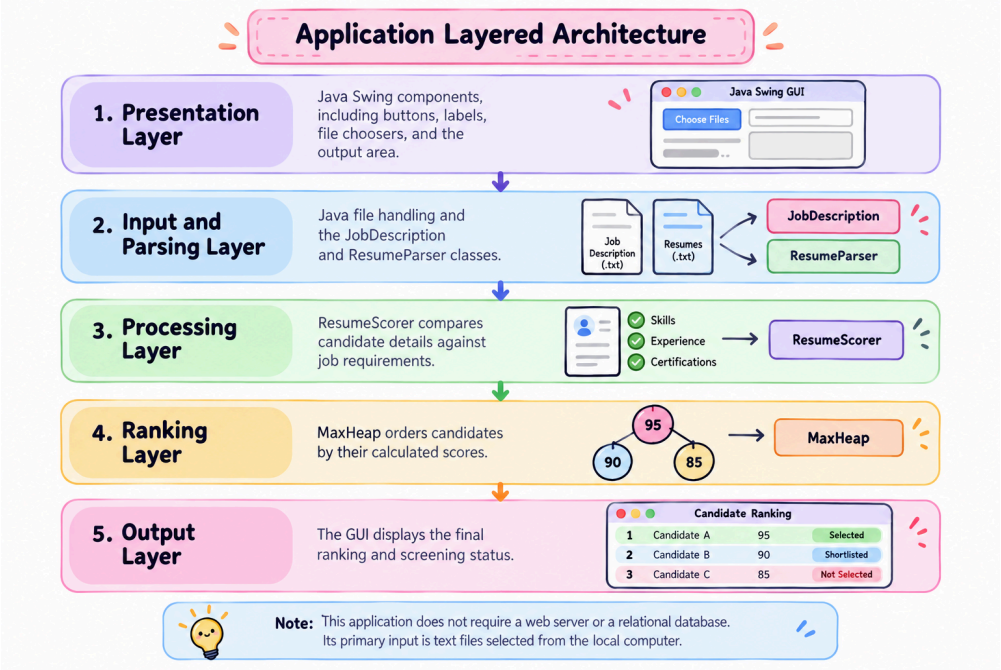


Figure 2.2 : System Architecture of Smart Resume Screening Tool

2.3 Job description processing module

The user selects a job description file using `JFileChooser`. The `JobDescription` class reads the file and provides the requirements used by the screening process, including required skills, minimum experience, and certifications.

The parsed requirements become the reference against which candidate resumes are evaluated.

2.4 Resume parsing module

The `ResumeParser` class reads each selected resume file and extracts the available candidate information. The information is represented using the `Candidate` class.

The candidate object contains:

- Name
- Skills
- Experience in years
- Certifications
- Calculated score

The accuracy of extraction depends on the structure and wording of the input resume. The current text-based approach may not reliably interpret every resume layout.

2.5 Skill matching and candidate scoring

The `ResumeScorer` class compares extracted candidate information with the job description requirements. It calculates a score that is used to determine the candidate's position in the final ranking.

The scoring process can consider skill matches, experience requirements, and certification matches according to the implemented scoring rules. A higher score indicates a stronger match according to those rules; it does not guarantee that a candidate is suitable in every respect.

2.6 Candidate ranking using max heap

A Max Heap is a binary heap in which the highest-priority element is maintained at the root. In this project, candidate scores determine priority.

The ranking process is:

1. Calculate the score for each candidate.
2. Insert each candidate into the custom Max Heap.
3. Extract the maximum-score candidate.
4. Display the candidate's rank and details.
5. Repeat until the heap is empty.

For n candidates, insertion of all candidates takes $O(n \log n)$ time in the worst case, and extracting all candidates takes $O(n \log n)$. Therefore, the overall heap-based ranking process takes $O(n \log n)$ time in the worst case.

2.7 GRAPHICAL USER INTERFACE

The interface is developed using Java Swing. It includes a job description selection button, a resume folder selection button, an Analyze Resumes button, and a results display.

The interface also validates whether the required inputs have been selected and provides feedback when the input folder contains no files or when processing encounters an error.

Chapter 3: Implementation

3.1 Software Requirements

S. No.	Software / Technology	Purpose
1	Java Development Kit (JDK)	Compiling and executing the application
2	Eclipse IDE	Writing, debugging, and running Java code
3	Java Swing	Developing the graphical user interface
4	Java File I/O	Reading resume and job description files
5	java.io.File	Accessing files in the selected resume folder
6	JFileChooser	Selecting the job description and resume folder
7	Custom Max Heap	Ranking candidates by score
8	Java Arrays and Classes	Storing and processing candidate information

Table 3.1 : Software Requirements

3.2 Hardware Requirements

S. No.	Component	Minimum / Recommended Specification
1	Processor	Intel Core i3 or equivalent
2	RAM	4 GB minimum; 8 GB recommended
3	Storage	Sufficient space for Eclipse, JDK, and project files
4	Operating System	Windows 10/11 or another Java-compatible OS

Table 3.2 : Hardware Requirements

3.3 GRAPHICAL USER IMPLEMENTATION

The graphical user interface is implemented using Java Swing. The main window contains a title, file selection controls, and an output area for the final screening results. The JFileChooser component enables the user to select a job description file and a folder containing candidate resumes.

When the user clicks the Analyze Resumes button, the application validates the selected inputs and initiates the screening process. The parsed candidate information is scored and inserted into the Max Heap. The application then extracts candidates in priority order and displays their rank, name, experience, score, and screening status.

The interface separates user interaction from the main processing logic, making the application easier to operate and demonstrate.

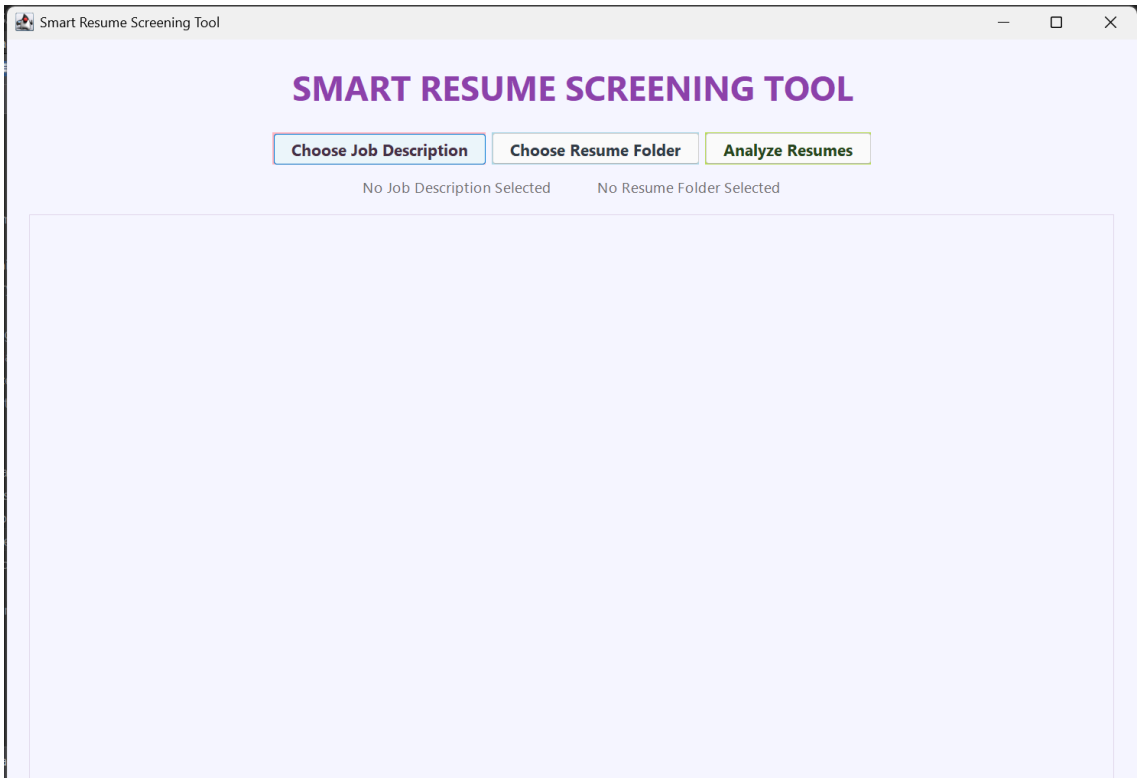


Figure 3.1 : Graphical User Interface of the Application

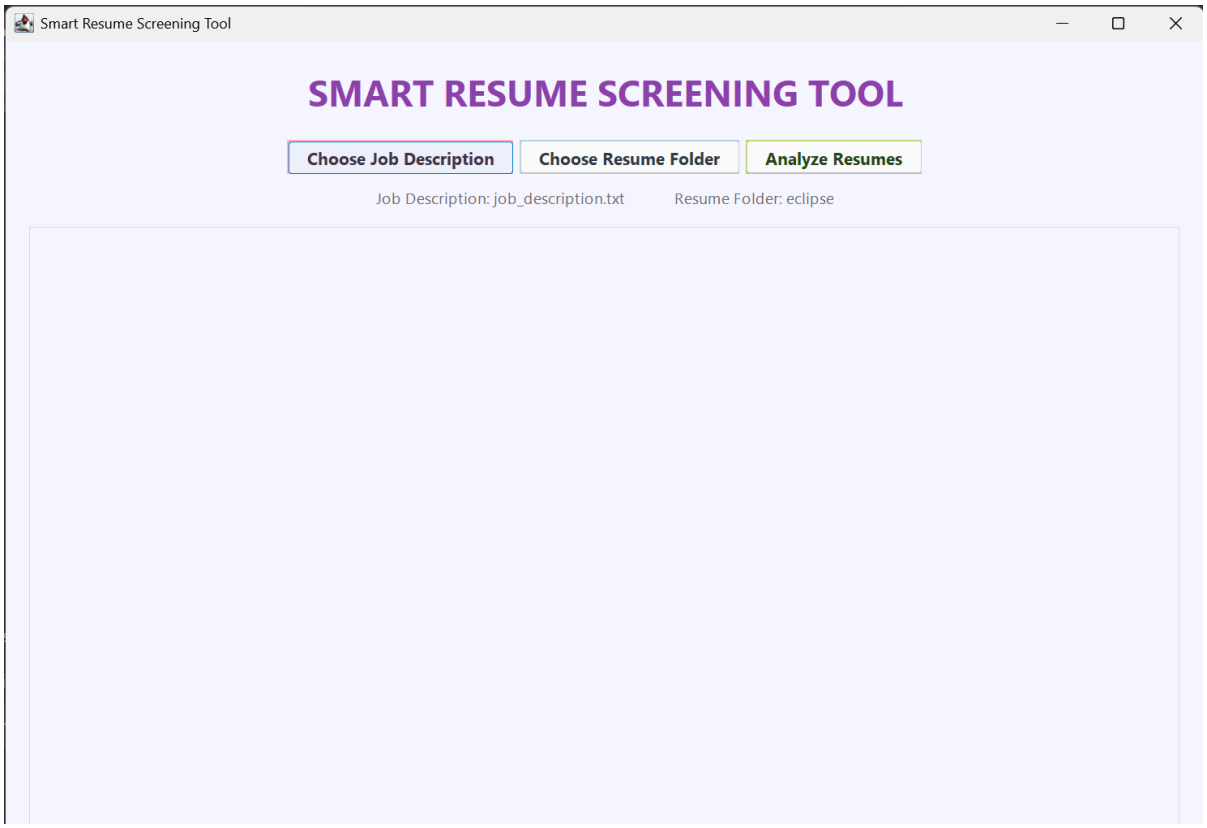


Figure 3.2 : Job Description File Selection

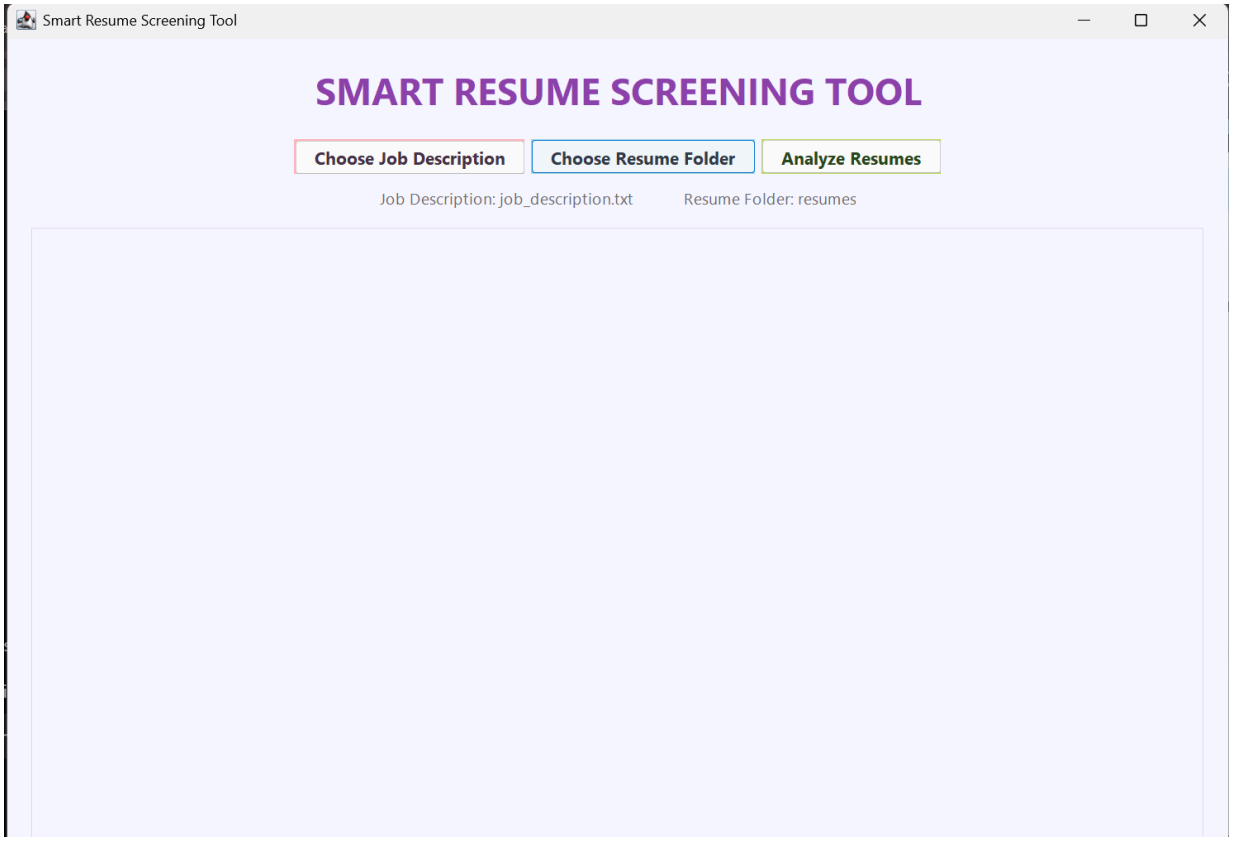


Figure 3.3 : Resume Folder Selection

3.4 Resume parsing implementation

The ResumeParser class reads the selected resume file and constructs a Candidate object containing the extracted information. The parser is responsible for identifying supported fields from the text representation of a resume.

The quality of the extracted information depends on the parser's implemented rules and the format of the resume. Different resume layouts may require additional parsing logic.

3.5 Candidate scoring implementation

The ResumeScorer class calculates a numerical score by comparing candidate details with the job description. The score is stored in the candidate object using its setScore() method.

The score is used as a ranking value, rather than a statistically calibrated probability of employment success. The precise score calculation should be documented using the weights and conditions in your actual ResumeScorer.java.

3.6 Max Heap implementation

The MaxHeap class maintains candidate priorities using their scores. The insert() method adds candidates to the heap, while extractMax() retrieves the highest-scoring candidate and restores the heap property.

This implementation demonstrates the use of a priority-based data structure for efficient ranking.

3.7 Algorithm Analysis

S. No.	Operation	Algorithm / Data Structure	Time Complexity
1	Read resume text	File I/O	Depends on total input size
2	Compare strings	Implemented string-matching algorithm	Depends on the algorithm used
3	Calculate candidate score	Scoring rules	Depends on the number of requirements
4	Insert one candidate	Binary Max Heap	$O(\log n)$
5	Extract highest-scoring candidate	Binary Max Heap	$O(\log n)$
6	Build the heap using repeated insertions	Binary Max Heap	$O(n \log n)$
7	Rank all candidates by repeated extraction	Binary Max Heap	$O(n \log n)$

Table 3.3 : Algorithms and Time Complexity

Chapter 4: Data and File Design

4.1 Job description file

The job description file provides the criteria for screening candidates.

Example structure:

Job Title: Software Developer

Job Description: We are looking for a Software Developer with strong knowledge of Java, Python, SQL, Data Structures, and Algorithms.

Preferred Skills: Spring Boot, AWS, MySQL

Experience: Candidates should have at least 2 years of experience.

AWS certification is preferred.

Responsibilities:

- Develop scalable applications
- Design database systems
- Work with cloud platforms

The exact labels and syntax should match what your JobDescription.java parser expects.

4.2 Resume file structure

Each resume is stored as a separate text file in the selected folder.

Example:

Name: Rahul Sharma

Email: rahulsharma@gmail.com

Phone: 9876543210

Location: Hyderabad

Career Objective: Motivated Computer Science graduate seeking a Software Developer position where I can apply my programming, problem-solving, and software development skills.

Education: B.Tech in Computer Science Engineering

Koneru Lakshmaiah University

CGPA: 8.9

Skills: Java, Python, SQL, Spring Boot, AWS, Data Structures, Algorithms, HTML, CSS, JavaScript

Experience: 2

Projects:

1. Smart Resume Screening Tool

Developed a resume analysis system using Java and DSA concepts.

Implemented skill matching and candidate ranking.

2. Student Management System

Built a database-driven application using Java and MySQL.

Certifications: AWS

Achievements:

- Solved 300+ coding problems on LeetCode.
- Participated in Smart India Hackathon.
- Completed Java Programming Certification.

Languages: English, Hindi, Telugu

Interests: Competitive Programming, Software Development, Artificial Intelligence

Use this as an example of a supported text format, not as a guarantee that every PDF, DOCX, or differently formatted resume will be parsed correctly.

4.3 Candidate data structure

The Candidate class stores the parsed information.

Field	Data type	Description
name	String	Candidate's name
skills	String[]	Extracted technical skills
experienceYears	int	Experience in years
certifications	String[]	Extracted certifications
score	int	Calculated screening score

Table 4.1 : Candidate Data Fields

The class provides getter methods to access candidate details and a setter method to update the score.

4.4 Max Heap design

The custom Max Heap is responsible for ordering candidates by score. The highest-scoring candidate is extracted first, followed by the next highest-scoring candidate.

For n candidates, the heap-based ranking process requires $O(n \log n)$ time in the worst case when candidates are inserted individually and then extracted one at a time.

No persistent database schema is included in the current implementation. Candidate objects exist in memory during screening, and the input files remain stored on the local computer.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 RESULTS OBTAINED

The Smart Resume Screening Tool provides a graphical workflow for selecting job requirements and a folder of resumes. The system processes the selected files, extracts supported candidate information, calculates candidate scores, and ranks the candidates using a Max Heap.

The application presents the ranked candidates through the GUI, displaying their names, years of experience, scores, and screening status. The use of a priority-based data structure enables candidates to be displayed in descending order of their calculated scores.

The results demonstrate how file handling, object-oriented programming, string processing, scoring rules, and heap-based ranking can be combined to address an initial resume screening problem.

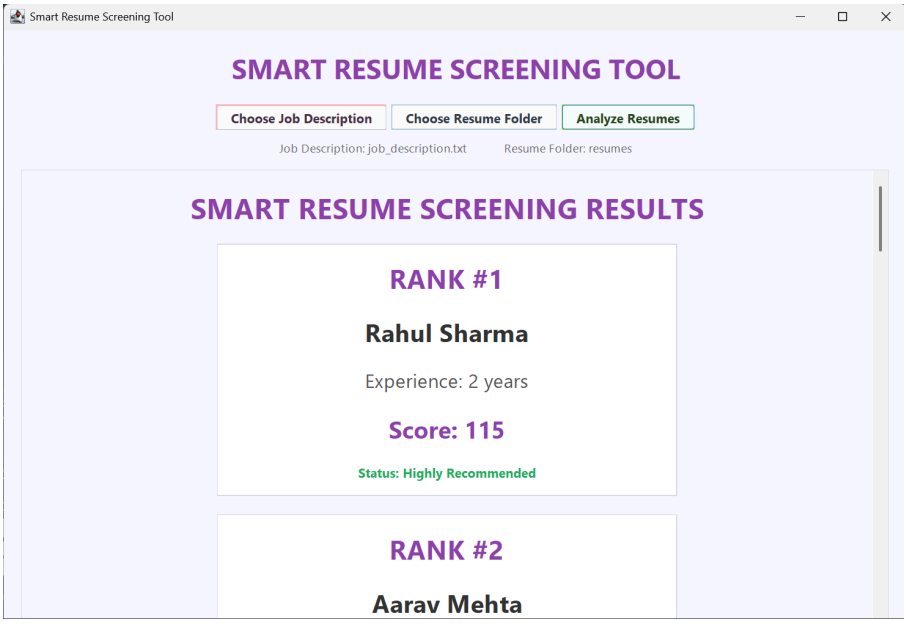


Figure 5.1 : Application Output

5.2 Discussion

The project demonstrates that resume screening can be organized as a sequence of parsing, scoring, and ranking operations. Candidate information is stored in structured objects, while the Max Heap provides an efficient way to retrieve candidates in score order.

The GUI makes the application easier to demonstrate because users can select input files rather than edit file paths in the source code. However, the quality of the ranking depends on the correctness of resume extraction and the design of the scoring rules. The tool should therefore be treated as decision support for recruiters rather than an automatic hiring authority.

5.3 Advantages

- Reduces repetitive manual comparison of resumes.
- Processes multiple candidate files in one screening run.
- Supports job-specific candidate evaluation.
- Uses a custom Max Heap for priority-based ranking.
- Provides a user-friendly Java Swing interface.
- Allows input selection through a file chooser.
- Demonstrates practical DSA concepts in a real-world-inspired application.

5.4 Limitations

- The current implementation is designed for text-based resume files.
- Extraction may fail when candidate details are presented in unexpected formats.
- Results depend on the implemented scoring rules and extracted data.
- The application does not currently provide database-backed storage or multi-user access.
- A higher score does not guarantee that a candidate is the best person for a role.
- Advanced PDF/DOCX extraction and semantic understanding require additional implementation.

5.5 Testing results

S. No.	Test Case	Expected Result	Status
1	Select job description	Selected file path is stored	Tested
2	Select resume folder	Selected folder path is stored	Tested
3	Click Analyze without selecting inputs	Validation message appears	Tested
4	Process a valid resume	Candidate details are extracted	Tested
5	Process multiple resumes	All supported resume files are processed	Tested

6	Calculate candidate scores	Scores are assigned according to configured rules	Tested
7	Rank candidates	Candidates are displayed in descending score order	Tested
8	Select an empty folder	A suitable message is displayed	Tested
9	Select an invalid or unreadable file	An error is handled without crashing the GUI	Tested

Table 5.1 : Test Cases and Results

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

The Smart Resume Screening Tool demonstrates the application of Data Structures and Algorithms to automate the initial screening of job applicants. The application is developed using Java and provides a graphical interface for selecting a job description and a folder of candidate resumes.

The system processes resume information, evaluates candidates according to job requirements, calculates screening scores, and ranks candidates using a custom Max Heap. The project combines file handling, string processing, object-oriented programming, and priority-based data organization in a practical application.

The implementation provides a foundation for improving the efficiency of preliminary candidate screening. Its effectiveness depends on the accuracy of resume parsing and the scoring criteria used. Further enhancements can improve file-format support, matching accuracy, and the overall usability of the system.

6.2 FUTURE SCOPE

The project can be enhanced through the following features:

1. PDF and DOCX support: Extract text from commonly used resume formats.
2. Improved skill extraction: Support different spellings, synonyms, and variations of technical skills.
3. Advanced string matching: Integrate algorithms such as KMP or Rabin–Karp where appropriate, and Edit Distance for approximate string matching.
4. Configurable scoring: Allow recruiters to adjust the importance of skills, experience, and certifications.
5. Database integration: Store candidate information and screening history in MySQL or PostgreSQL.
6. Improved graphical interface: Display candidate cards, match indicators, and sortable results.
7. Export functionality: Generate a report containing ranked candidates.
8. Larger-scale screening: Optimize parsing and ranking for larger resume collections.
9. Fairness and transparency: Provide explanations for candidate scores and allow human review of screening results.

References

Use references that support the technologies and algorithms actually used in your project.

1. Oracle, Java Platform Documentation. https://docs.oracle.com/en/java/
2. Oracle, Java Swing Tutorial. https://docs.oracle.com/javase/tutorial/uiswing/
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th Edition, MIT Press, 2022.
4. R. Sedgewick and K. Wayne, Algorithms, 4th Edition, Addison-Wesley, 2011.
5. Oracle, Java I/ODocumentation. https://docs.oracle.com/javase/tutorial/essential/io/